

CIMA0_Core 顶层原则补充

原则 21：模块自治边界

每个模块：

只管理自己的内部状态。

不负责：

- 理解其他模块
- 管理其他模块
- 优化整体
- 维持全局目标

PlanetEngine

只知道：

x
v
omega
自身动力规则

不知道：

- Cloud存在
- Observer存在
- Compute存在
- IO来源

它可以接收扰动。

但是：

是否响应，由 Planet 自己的动力规则决定。

不是外部强制注入。

Cloud

只知道：

自身云状态
自身演化规则

不知道：

- 谁读取它
- 谁需要它

- 哪个结构重要

它不会：

```
cloud.push_to_planet()
```

因为这代表 Cloud 理解外部。

Observer

只知道：

收到的局部信息
自己的历史

不知道：

- 世界总量
- 所有对象
- 哪个最高级

它只是：

观察

↓

产生信号

Compute

只知道：

当前计算任务
自身资源

不知道：

- 为什么任务出现
 - 世界状态
 - 结构意义
-

IO

只知道：

bytes

不解释。

交互方式修正

错误：

A模块主动控制B模块

例如：

cloud.force(planet)

删除。

正确：

外部事件

↓

多个模块各自接触

↓

各自响应

↓

产生新的状态

类似自然系统：

风吹过森林

树不知道风是什么

风也不知道树是什么

树根据自身结构响应

因此后续代码要避免：

禁止：

```
main.py

cloud.update()
planet.update(cloud)
observer.update(planet)
compute.update(observer)
```

这种中心调度关系。

因为 main 变成了上帝视角。

更接近：

```
main

只提供时间流逝

↓

各模块自行step()

↓

各模块自行决定是否响应
```

例如：

```
for t in range():

    io.step()

    cloud.step()

    planet.step()

    observer.step()

    compute.step()
```

每个：

不知道其他模块。

所以刚才 Cloud 接口设计需要重新调整。

不是：

Cloud提供力给Planet

而是：

Cloud产生自身变化

Planet如果处在某种耦合条件下，自身选择响应

也就是说：

耦合关系不能由一个模块拥有。

应该属于更底层的共同规则。

这个原则比之前的 Cloud/Cell 讨论更基础。

后续代码检查顺序：

1. 每个模块有没有管理自己以外的东西？
2. 有没有读取全局？
3. 有没有主动控制其他模块？
4. 有没有隐藏中心调度？

符合后，才讨论功能。